

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «АПО ЭВМиС»
**Изучение аппаратно-программной архитектуры процессоров семейства
Intel – разработка ассемблерной вставки**

Выполнил:
Студент 4-го курса
Группы АС-66
Пекун М.С.
Проверил:
Кулик А.Д.

Брест 2026

Цель и задание

Цель работы — изучить регистры и адресацию памяти процессоров Intel и применить встроенный ассемблер Microsoft Visual C++ для обработки строк. Сравнить результат и быстродействие с реализацией на C++.

Вариант 3: даны четыре строки. Поменять местами 1-ю с 3-й и 2-ю с 4-й. Исходный порядок (S1, S2, S3, S4) должен преобразоваться в (S3, S4, S1, S2).

Описание решения

Программа считывает четыре строки в отдельные буферы по 256 байт, включая завершающий нуль. Функция `swap_cpp` обменивает соответствующие байты двух пар буферов средствами C++. Функция `swap_asm` выполняет тот же обмен в цикле с регистрами ESI, EDI, ECX, AL и AH. Оба варианта работают с независимыми копиями исходного набора. После вызова результаты сравниваются побайтово.

Для измерения использована конфигурация Release Win32. В каждой из пяти серий функция вызывается 200 000 раз; в отчёт выводится медиана длительности серии. Копирование исходных данных и вывод на экран в измеряемый интервал не входят. Поскольку обмен повторяется чётное число раз, по окончании серии буферы возвращаются к исходному порядку.

Текст программы

Файл `string_swap.h`

```
#pragma once

#include <cstdint>

constexpr std::size_t kStringCapacity = 256;

struct FourStrings {
    char line[4][kStringCapacity]{};
};

__declspec(noinline) void swap_cpp(FourStrings& strings);
__declspec(noinline) void swap_asm(FourStrings& strings);
```

Файл `string_swap.cpp`

```
#include "string_swap.h"

#include <utility>

static_assert(sizeof(FourStrings) == 4 * kStringCapacity,
    "The assembly offsets require four adjacent buffers.");

void swap_cpp(FourStrings& strings) {
    for (int index = 0; index < static_cast<int>(kStringCapacity); ++index) {
        std::swap(strings.line[0][index], strings.line[2][index]);
        std::swap(strings.line[1][index], strings.line[3][index]);
    }
}

void swap_asm(FourStrings& strings) {
    FourStrings* data = &strings;
```

```

__asm {
    mov esi, data
    xor edx, edx
    mov ecx, 256
swap_next_byte:
    // Exchange the corresponding bytes of strings 1 and 3.
    mov al, byte ptr [esi + edx]
    mov ah, byte ptr [esi + edx + 512]
    mov byte ptr [esi + edx], ah
    mov byte ptr [esi + edx + 512], al

    // Exchange the corresponding bytes of strings 2 and 4.
    mov al, byte ptr [esi + edx + 256]
    mov ah, byte ptr [esi + edx + 768]
    mov byte ptr [esi + edx + 256], ah
    mov byte ptr [esi + edx + 768], al

    inc edx
    loop swap_next_byte
}
}

```

Файл main.cpp

```

#include "string_swap.h"

#include <algorithm>
#include <chrono>
#include <cstring>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

namespace {

constexpr int kIterations = 200000;
constexpr int kTrials = 5;
volatile unsigned int g_observed_byte = 0;

bool read_four_strings(FourStrings& result) {
    for (int index = 0; index < 4; ++index) {
        for (;;) {
            std::cout << "Enter string " << index + 1 << " (up to 255 bytes): ";
            std::string input;
            if (!std::getline(std::cin, input)) {
                return false;
            }
            if (input.size() >= kStringCapacity) {
                std::cout << "Too long. Please enter this string again.\n";
                continue;
            }
            std::memcpy(result.line[index], input.c_str(), input.size() + 1);
            break;
        }
    }
    return true;
}

void print_four_strings(const char* label, const FourStrings& strings) {
    std::cout << label << '\n';
    for (int index = 0; index < 4; ++index) {
        std::cout << " " << index + 1 << ": [" << strings.line[index] << "]\n";
    }
}

double median_milliseconds(void (*operation)(FourStrings&),
                           const FourStrings& original) {
    using clock = std::chrono::steady_clock;
    std::vector<double> samples;
    samples.reserve(kTrials);
}

```

```

for (int trial = 0; trial < kTrials; ++trial) {
    FourStrings working = original;
    operation(working); // Warm up the function before each measured series.
    working = original;

    const auto start = clock::now();
    for (int iteration = 0; iteration < kIterations; ++iteration) {
        operation(working);
    }
    const auto finish = clock::now();
    g_observed_byte += static_cast<unsigned char>(working.line[0][0]);
    samples.push_back(
        std::chrono::duration<double, std::milli>(finish - start).count());
}

std::sort(samples.begin(), samples.end());
return samples[kTrials / 2];
}

} // namespace

int main() {
    FourStrings original;
    if (!read_four_strings(original)) {
        std::cerr << "Input ended before all four strings were entered.\n";
        return 1;
    }

    FourStrings cpp_result = original;
    FourStrings asm_result = original;
    swap_cpp(cpp_result);
    swap_asm(asm_result);

    print_four_strings("Original strings:", original);
    print_four_strings("After C++ function:", cpp_result);
    print_four_strings("After ASM function:", asm_result);

    if (std::memcmp(&cpp_result, &asm_result, sizeof(FourStrings)) != 0) {
        std::cerr << "ERROR: C++ and ASM results differ.\n";
        return 2;
    }
    std::cout << "Results match.\n";

    const double cpp_ms = median_milliseconds(swap_cpp, original);
    const double asm_ms = median_milliseconds(swap_asm, original);
    std::cout << std::fixed << std::setprecision(3);
    std::cout << "Release Win32 benchmark: " << kTrials << " trials, "
        << kIterations << " swaps per trial (median).\n";
    std::cout << "C++: " << cpp_ms << " ms\n";
    std::cout << "ASM: " << asm_ms << " ms\n";
    if (asm_ms > 0.0) {
        std::cout << "C++ / ASM time ratio: " << cpp_ms / asm_ms << '\n';
    }
    return 0;
}

```

Результаты выполнения

Для демонстрационного запуска введены строки Alpha, B, Third line и Fourth. В обоих вариантах результат — Third line, Fourth, Alpha, B. Программа подтвердила побайтовое совпадение буферов. Ниже показан зафиксированный вывод этого запуска.

```
Lab1Variant3.exe | Release Win32

Original strings:
 1: [Alpha]
 2: [B]
 3: [Third line]
 4: [Fourth]
After C++ function:
 1: [Third line]
 2: [Fourth]
 3: [Alpha]
 4: [B]
After ASM function:
 1: [Third line]
 2: [Fourth]
 3: [Alpha]
 4: [B]
Results match.
Release Win32 benchmark: 5 trials, 200000 swaps per trial (median).
C++: 125.556 ms
ASM: 130.864 ms
C++ / ASM time ratio: 0.959
```

Рисунок 1 — Вывод программы при запуске Release Win32

Реализация	Медиана времени, мс
C++	125.556
Встроенный ASM	130.864

Вывод

Обе функции выполнили вариант 3 правильно и дали одинаковые результаты. В данном запуске быстрее оказалась реализация на C++: C++ — 125.556 мс, ASM — 130.864 мс за 200 000 обменов. Отношение времени C++ к ASM составило 0.959. Это измерение характеризует именно данную конфигурацию сборки и компьютер; ассемблерная вставка сама по себе не гарантирует ускорения по сравнению с оптимизированным C++.